

Vier Architektur-Vordenker im Detail: Hohpe · Fowler · Martin · Hohmann

Werke, Kernkonzepte und was sie für Lagebild, Stabsarbeit und unsere Software hergeben

Version 1.0 · Stand 07.08.2026 · Zielgruppe: Portfolio- und Solution-Verantwortliche sowie Architekten · Nebenschrift zur Team-of-Teams-Denkschrift (v3.0, Teil C.4) und zur EA-Denkschrift (v2.0, Teil A2)

Zu diesem Dokument: Kuratiert, inhaltlich durchdacht und verantwortet von **Robert Seebauer**; recherchiert und erstellt mit Unterstützung von **Claude** (Anthropic – Modell **Claude Fable 5**, via Claude Code). Dies ist **Version 1.0** (Erstfassung). Die Kurzfassungen dieser vier Profile stehen in der Team-of-Teams-Denkschrift v3.0 (Teil C.4, Stabs-/Führungsrelevanz) und der EA-Denkschrift v2.0 (Teil A2, EA-Übertragung); diese Nebenschrift ist die ausführliche Ausarbeitung. Versionsführung: `Quellen/Versionierung.md` bzw. `Quellen/Versionen/`.

Executive Summary

Ausgangsfrage: Vier Praxis-Vordenker der Software-Architektur – Gregor Hohpe, Martin Fowler, Robert C. Martin, Luke Hohmann – tragen die Denkschriften-Reihe als „zivile Kronzeugen“. Was steht *im Detail* in ihren Werken, wie hängen die Konzepte zusammen – und was folgt konkret für Lagebild, Stabsausbildung und die SituationReport-Software?

WICHTIGSTER EINZELSATZ DES DOKUMENTS

Vier Wege, ein Ziel: Architekturarbeit ist **Entscheiden unter Unsicherheit mit gemeinsamer Sprache** – ihre Werkzeuge (Muster, Optionen, Grenzen, Spiele) sind darum **Führungs- und Stabswerkzeuge**, keine Technik-Interna.

Befund in fünf Sätzen:

- 1 Alle vier lösen dasselbe Übersetzungsproblem** – Hohpe fährt den Aufzug zwischen Vorstand und Maschinenraum, Fowler und Martin geben den Ebenen eine gemeinsame Mustersprache, Hohmann bringt beide an einen Spieltisch. Das Rosetta-Stone-Problem (EA-Denkschrift, Befund 3) ist ihr gemeinsamer Nenner.
- 2 Mustersprachen sind Doktrin:** 65 Integrationsmuster (Hohpe/Woolf) und 51 Enterprise-Muster (Fowler) leisten zivil, was MDMP und Planungssprache militärisch leisten – zusammengewürfelte Teams sind sofort arbeitsfähig, weil die Begriffe gelehrt und geteilt sind.
- 3 Gute Architektur hält Optionen offen:** Hohpes „first derivative“/Real-Options-Denken und Martins Dependency Rule („maximiere die Zahl der noch nicht getroffenen Entscheidungen“) sind zwei Formulierungen desselben Prinzips – Clausewitz' Umgang mit Friktion, in Software gegossen.
- 4 Beteiligung erzeugt Legitimität und Lagebild:** Hohmanns ernsthafte Spiele (Buy a Feature, Participatory Budgeting) sind Entscheidungs-Rituale, in denen die Beteiligten die Trade-offs selbst sehen und tragen – Shared Consciousness und Empowered Execution in einem Format.
- 5 Konsequenz für unsere Software:** Vier direkt umsetzbare Ableitungen – Trend-/Ableitungsspalten je Kennzahl (Hohpe), Schulden-Quadrant im NFR-/Schulden-Register (Fowler), Strangler-Fortschrittsmetrik für Modernisierungs-Epics (Fowler), Profit-Stream-Feld je Capability (Hohmann) – alle anschlussfähig an die Phasen A–C der Roadmap in `docs/` (Teil E.3).

Teil A – Gregor Hohpe: der Übersetzer zwischen den Stockwerken

Person und Werklinie: Hohpe hat die Vermittlerrolle, über die er schreibt, selbst in Serie gelebt: Chefarchitekt der Allianz, Technical Director im CTO-Office von Google Cloud, Enterprise Strategist bei AWS, Smart-Nation-Fellow in Singapur. Seine Werke bauen aufeinander auf: *Enterprise Integration Patterns* (mit Bobby Woolf, 2003) gab der Integrationstechnik ihre Sprache; *The Software Architect Elevator* (2020) definierte die Rolle des Architekten neu; *Cloud Strategy* (2020) und *Platform Strategy* (2024, mit Danieli/Landreau) übertragen beides auf die zwei größten Strukturentscheidungen der Gegenwart. Laufend ergänzt auf architectelevator.com.

A.1 The Software Architect Elevator – die Rolle

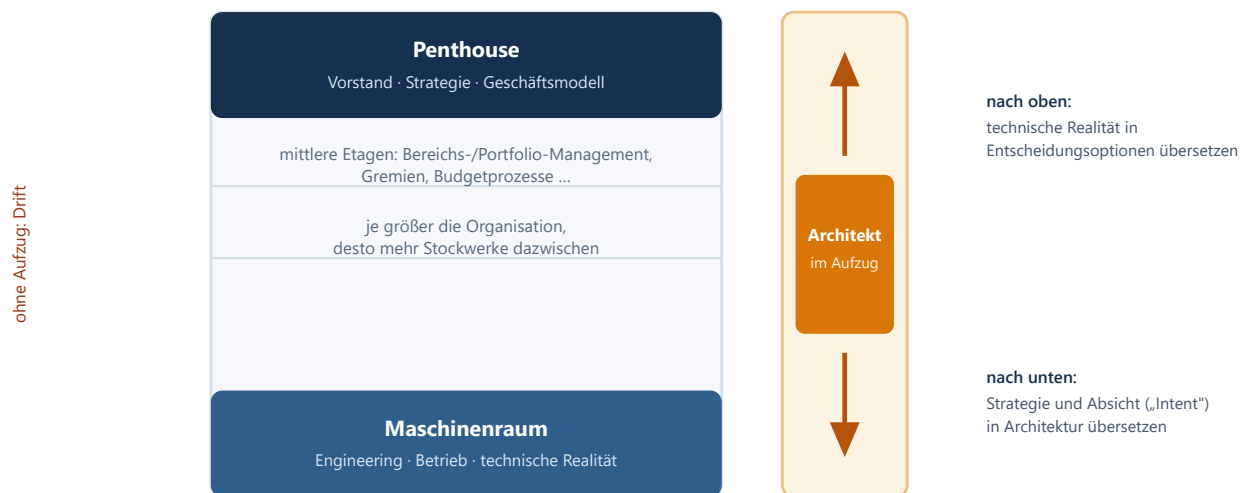


Abb. 1 – Der Architect Elevator: Wert entsteht durch die Fahrt, nicht durch das Stockwerk. Ohne Aufzugfahrer driften Penthouse und Maschinenraum auseinander, ohne es zu merken.

A.1.1 Die Kernideen des Elevator-Buchs

- **Die Fahrt ist der Wert.** Architekten, die nur im Penthouse präsentieren, werden zu Folien-Architekten; wer nur im Maschinenraum bleibt, überlässt Entscheidungen denen ohne technische Realität. Beides zusammen ist die Rolle – und sie skaliert mit der Zahl der Stockwerke (Konzerngröße).
- „*Architects live in the first derivative.*“ Der Zustand eines Systems sagt wenig; entscheidend ist, **wie schnell und in welche Richtung** es sich ändert. Architektur bewertet Änderungsraten – und schafft Strukturen, die Änderungsraten erhöhen (Entkopplung, Automatisierung, Feedback).
- **Architektur verkauft Optionen.** In Anlehnung an Finanz-Optionen: Eine gute Architekturentscheidung kauft das Recht (nicht die Pflicht), später billiger reagieren zu können – z. B. Skalierungsoption, Exit-Option, Deployment-Option. Der Preis der Option (Abstraktion, Aufwand) ist gegen die Unsicherheit abzuwägen: **je unsicherer die Zukunft, desto wertvoller die Option.**
- **Organisationen sind Systeme.** Hohpe wendet Systemdenken auf die IT-Organisation selbst an: Ohne Feedbackschleifen (Lagebild!) regelt niemand nach; Umkehrschluss: Wer Strukturen ändern will, muss die Regelkreise ändern, nicht die Folien.

A.2 Enterprise Integration Patterns – die Sprache der Integration

65 Muster mit Namen, Problem, Kontext und Lösung – seit 2003 *die* Referenzsprache für asynchrone Integration; Messaging-Plattformen von Kafka über RabbitMQ bis zu iPaaS-Produkten verwenden dieses Vokabular bis heute wörtlich. Für die Solution-Ebene (mehrere ARTs, eine Lösung) ist Integration die Kernrealität – die wichtigsten Muster im Überblick:

Muster (EIP)	Was es löst	Relevanz für die Solution-Ebene
Publish-Subscribe vs. Point-to-Point Channel	Grundentscheidung: ein Empfänger oder alle Interessenten?	Entkopplungsgrad zwischen ARTs – bestimmt, wer bei Änderungen mitbetroffen ist
Message Translator / Canonical Data Model	Übersetzen zwischen Datenmodellen; gemeinsames Kernmodell statt n×n-Übersetzungen	Das Datenmodell-Gegenstück zur Capability-Map: eine gemeinsame Sprache der Fachobjekte
Content-Based Router / Message Filter	Nachrichten anhand des Inhalts lenken bzw. aussortieren	Verantwortungsschnitte sichtbar machen: Wer entscheidet, wohin Arbeit fließt?
Splitter / Aggregator / Scatter-Gather	Große Aufträge zerlegen, Teilergebnisse wieder zusammenführen	Exakt das Muster des aggregierten Situation Reports: viele Team-Beiträge → ein Bild
Correlation Identifier	Zusammengehörige Nachrichten über Systeme hinweg wiedererkennen	End-to-End-Nachverfolgbarkeit über ARTs – Voraussetzung für ehrliche Lead-Time-Messung
Dead Letter Channel / Guaranteed Delivery	Was passiert mit unzustellbaren Nachrichten?	Die unbequemen Fragen der Integrationsrealität – gehören als Risiko ins Lagebild
Idempotent Receiver / Competing Consumers	Doppelte Zustellung unschädlich machen; Last auf mehrere Verarbeiter verteilen	Robustheits-Grundwissen (vgl. EA-Denkschrift C.2: CAP, Eventual Consistency)
Control Bus	Die Integrationslandschaft selbst überwachen und steuern	Das EIP-eigene Plädoyer fürs Lagebild: Monitoring als eingebautes Muster, nicht als Nachrüstung

A.3 Cloud Strategy & Platform Strategy – Strukturentscheidungen als Strategie

A.3.1 Cloud Strategy (2020)

Cloud-Migration ist eine **Entscheidungsdisziplin**, keine Technikübung: Das Buch ist als Sammlung von Entscheidungsmodellen aufgebaut (Migrationspfade wie Lift-and-Shift vs. Re-Platform vs. Re-Architect, deren echte Kosten und Optionswerte), mit der Kernwarnung, dass nachgeplapperte Strategie-Floskeln („Cloud first!“) keine Entscheidungen ersetzen. Übertragung: Dieselbe Disziplin – Optionen explizit, Kriterien vorab, Annahmen dokumentiert – verlangt unsere Denkschriften-Reihe für jede große Portfolio-Entscheidung (MDMP, Modul 2).

A.3.2 Platform Strategy (2024)

„**Innovation durch Harmonisierung**“: Interne Plattformen sind kein Sparprogramm, sondern verschieben die Build/Buy-Linie – Teams bauen weniger Undifferenziertes selbst und innovieren schneller *auf* der Plattform. Gelingensbedingungen: Plattform als Produkt führen (freiwillige Nutzung schlägt Zwang), Harmonisierung von unten statt Standardisierung per Dekret, und ehrliche Metriken über Adoption statt Compliance. Übertragung: Die Plattform *ist* operationalisierter Architecture Runway – ihr Zustand (Adoption, Lücken, Schulden) gehört als Runway-Dimension ins Solution-Lagebild.

BEDEUTUNG FÜR LAGEBILD & STABSARBEIT

(1) Die **vertikale Liaison** wird Rolle im Rollen-Mapping (ToT C.1) – mit den Besten besetzt, karrierewirksam. (2) Das Lagebild zeigt **Ableitungen**: Trends, Beschleunigungen, Kippunkte je Kennzahl – nicht nur Zustände. (3) Architektur-Entscheidungsvorlagen weisen den **Optionswert** aus (welche Zukunftsoptionen kauft/verbaut diese Entscheidung?). (4) Integrationsrealität wird im **EIP-Vokabular** berichtet – benannte Muster statt Prosa.

Teil B – Martin Fowler: der Evolutionist

Person und Werkinie: Fowler ist seit 2000 Chief Scientist bei Thoughtworks, Mitunterzeichner des Agilen Manifests (2001) und betreibt mit **martinfowler.com** (dem „bliki“) das wohl einflussreichste lebende Referenzwerk der Softwareentwicklung. Seine Bücher *Refactoring* (1999; 2. Aufl. 2018) und *Patterns of Enterprise Application Architecture* (2002) haben zwei Ideen im Mainstream verankert: Software-Design ist **evolutionär verbesserbar**, und Entwurfswissen wird über **benannte Muster** weitergegeben.

B.1 Refactoring – Struktur verbessern, ohne Verhalten zu ändern

B.1.1 Das Handwerk

Refactoring ist definiert als **verhaltenserhaltende Strukturverbesserung in kleinen, abgesicherten Schritten** – ein Katalog benannter Umbauten (Extract Function, Move Field, Replace Conditional with Polymorphism ...), jeder klein genug, um zwischendurch immer lauffähig zu bleiben. Zwei Konsequenzen: Refactoring ist **Teil der täglichen Arbeit**, kein „Refactoring-Projekt“; und es setzt Tests voraus – ohne Sicherheitsnetz ist Umbau Glücksspiel. Die 2. Auflage (2018) modernisierte den Katalog und die Beispiele.

B.1.2 Design-Stamina-Hypothese & Technische-Schulden-Quadrant

Die **Design-Stamina-Hypothese** (bliki): Ohne Investition in innere Qualität sinkt die Feature-Geschwindigkeit mit wachsender Codebasis rapide – gute Architektur zahlt sich nicht „irgendwann“, sondern bereits nach Wochen aus. Fowlers Antwort auf „Können wir uns Qualität leisten?“ ist die Umkehrung: „*High quality software is cheaper to produce.*“ Der **Technische-Schulden-Quadrant** macht Schulden steuerbar statt moralisch: bewusst/unbewusst × klug/leichtsinnig – die kluge, bewusste Schuld („wir liefern jetzt und zahlen dokumentiert zurück“) ist legitimes Werkzeug; gefährlich ist die unbewusste, leichtsinnige.



Abb. 2 – Fowlers Technische-Schulden-Quadrant: Schulden werden steuerbar, wenn man ihre Entstehungsart benennt – im Lagebild klassifizieren statt moralisieren.

B.2 Patterns of Enterprise Application Architecture – das Vokabular

51 Muster für Unternehmensanwendungen, gegliedert nach Schichten – bis heute die Referenzsprache, in der Frameworks ihre Konzepte benennen. Die für Solution-Diskussionen wichtigsten:

Muster (PoEAA)	Kernidee und heutige Relevanz
Domain Model vs. Transaction Script	Die Grundsatzentscheidung: reiche Fachlogik als Objektmodell oder prozedurale Abläufe? Bestimmt Wartbarkeit bei wachsender Komplexität
Service Layer	Definierte Anwendungsgrenze mit klaren Operationen – der Vorläufer heutiger API-First-Schnitte
Repository / Data Mapper	Fachmodell von Persistenz entkoppeln – Grundlage jeder testbaren, austauschbaren Datenzugriffsschicht (jedes ORM spricht dieses Vokabular)
Unit of Work / Identity Map / Lazy Load	Konsistente Änderungsverfolgung und Ladeverhalten – die Begriffe hinter jedem ORM-Verhalten, das Teams „magisch“ vorkommt
Gateway / Remote Facade / DTO	Fremdsysteme kapseln, Netzwerkgrenzen grob-granular schneiden – das Handwerkszeug der Systemgrenzen
Optimistic/Pessimistic Offline Lock	Nebenläufigkeit über Benutzer-Sessions hinweg – der Klassiker hinter „verlorenen Updates“

B.3 Strangler Fig & die bliki-Schlüsselartikel

B.3.1 Strangler Fig Application (2004)

Benannt nach der Würgefeige, die ihren Wirtsbaum allmählich umwächst: Ein neues System wächst **Stück für Stück um das Altsystem**, übernimmt Funktion für Funktion (per Fassade/Routing umgeleitet), bis das Altsystem absterben kann. Der Gegenentwurf zum Big-Bang-Rewrite – jederzeit lieferfähig, Risiko pro Schritt klein, Fortschritt messbar (Anteil umgeleiteter Funktionalität). Heute das Standardmuster jeder Legacy-Modernisierung.

B.3.2 Weitere Schlüsselartikel (Auswahl fürs Briefing)

- **MonolithFirst / MicroservicePrerequisites:** Verteilte Architektur ist kein Startpunkt, sondern verdient zu werden (Betriebsreife, Monitoring, schnelle Deployments) – Warnung vor Architektur-Mode.
- **BranchByAbstraction & FeatureToggles:** Große Umbauten am lebenden System ohne Dauer-Branches – die Techniken hinter kontinuierlicher Lieferung.
- **YAGNI** („You Aren't Gonna Need It“): Spekulative Generalität ist Schuldenaufnahme ohne Gegenwert – das Korrektiv zum Options-Denken: Optionen kaufen ja, Paläste auf Verdacht bauen nein.
- **Is High Quality Software Worth the Cost?** (2019): die ökonomische Kurzfassung der Design-Stamina-These – Pflichtlektüre für Budget-Diskussionen.

BEDEUTUNG FÜR LAGEBILD & STABSARBEIT

(1) **Schulden-Quadrant als Berichtsformat:** Das NFR-/Schulden-Register klassifiziert jede Position nach Entstehungsart – das entgiftet die Diskussion und macht Tilgung planbar. (2) **Strangler-Fortschritt als Roadmap-Metrik:** Modernisierungs-Epics berichten „Anteil umgeleiteter Funktionalität“ statt „% fertig“. (3) **Muster-Vokabular als Doktrin:** der zivile Beleg für ToT-Erfolgsfaktor 4 – gemeinsame Sprache macht zusammengewürfelte Stäbe sofort arbeitsfähig.

Teil C – Robert C. Martin: Grenzen, Regeln, Handwerksethos

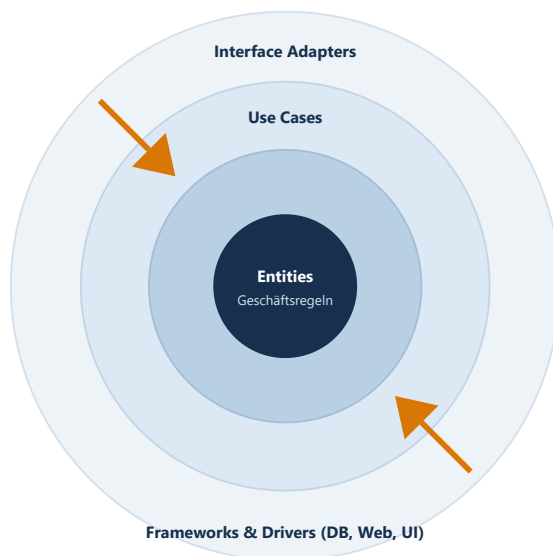
Person und Werklinie: „Uncle Bob“ Martin, seit den 1970ern Entwickler, Mitunterzeichner des Agilen Manifests und die prägende Stimme der **Software-Craftsmanship-Bewegung**. Er hat die SOLID-Prinzipien gebündelt und popularisiert und mit *Clean Code* (2008) und *Clean Architecture* (2017) zwei der meistgelesenen Bücher der Branche geschrieben – bewusst normativ formuliert, was ihre Stärke und ihre Schwäche ist (C.4).

C.1 Clean Code – Lesbarkeit als Ökonomie

C.1.1 Die Kernideen

- **Code wird ~10× öfter gelesen als geschrieben.** Deshalb ist Lesbarkeit keine Ästhetik, sondern der größte Kostenhebel: sprechende Namen, kleine Funktionen mit einer Aufgabe, keine Überraschungen.
- **Boy-Scout-Regel:** Den Code stets etwas sauberer hinterlassen, als man ihn vorfand – Qualität als kontinuierliche Kleinst-Investition statt Großprojekt (deckungsgleich mit Fowlers Refactoring-Alltag).
- **Tests als erste Bürger:** Testcode verdient dieselbe Sorgfalt wie Produktivcode; disziplinierte Entwicklung (bis hin zu TDD) ist Übungssache, nicht Talentfrage.

C.2 Clean Architecture – die Dependency Rule



Die Dependency Rule:

Quellcode-Abhängigkeiten zeigen ausschließlich nach innen – Geschäftsregeln wissen nichts von DB, Web, UI.

Konsequenzen:

- Frameworks/DB/UI sind *Details* – austauschbar, testbar, aufschiebbar
- Gute Architektur **maximiert die Zahl der noch nicht getroffenen Entscheidungen** (= Optionen offen)
- „Screaming Architecture“: Struktur schreit den Zweck, nicht das Framework

SOLID (Kurzform):

Single Responsibility · Open/Closed ·
Liskov Substitution · Interface Segregation ·
Dependency Inversion

Abb. 3 – Clean Architecture: konzentrische Ringe, Abhängigkeiten nur nach innen. Grenzen („Boundaries“) sind die teuersten und wertvollsten Entscheidungen – dieselbe Logik gilt für Solution- und ART-Schnitte.

C.2.1 Was davon auf die Solution-Ebene übertragbar ist

- **Boundaries sind Investitionsentscheidungen:** Wo eine Grenze verläuft (zwischen Services, ARTs, Zulieferern), bestimmt, was später billig oder teuer änderbar ist – die Dependency Rule ist das Prüfkriterium: Zeigen die Abhängigkeiten zur Fachlichkeit oder zur Technik?
- **„Details“ aufschieben:** Datenbank-, Framework- und UI-Festlegungen so spät wie verantwortbar treffen – wörtlich Hohpes Options-Verkauf, aus der Code-Perspektive formuliert.
- **Screaming Architecture als Review-Frage:** Erkennt man am Schnitt einer Solution ihren Geschäftszweck – oder nur ihre Technologiegeschichte?

C.3 Craftsmanship: Katas, Dojos, Disziplin

Die Craftsmanship-Bewegung (um Martin, Dave Thomas u. a.) hat **bewusstes Üben institutionalisiert**: **Code-Katas** – kleine, wiederholt trainierte Aufgaben – und **Coding Dojos** – Gruppenübungen mit Feedback – machen aus Prinzipien Reflexe. Martins Leitsatz „*The only way to go fast is to go well*“ ist die Kurzform: Disziplin ist Geschwindigkeitsvoraussetzung. Genau das lehrt Generalstabsausbildung seit 200 Jahren – Planübungen, bis die Form sitzt. Die Übertragung ins Curriculum (ToT Teil D) sind **Planungs-Katas**:

Planungs-Kata	Dauer	Trainierte Fertigkeit
Lageanalyse-Kata	30 Minuten	Aus einem Roh-Report Problem, Rahmen, Mittel und Annahmen extrahieren (MDMP Schritt 2)
Premortem-Kata	60 Minuten	„Es ist 12 Monate später, das Vorhaben ist gescheitert – warum?“ moderieren
Optionen-Kata	45 Minuten	Zu einem Vorhaben zwei echte Alternativen samt Vergleichskriterien entwickeln
BLUF-Kata	15 Minuten	Eine Lage in 5 Sätzen vortragen: Kernaussage zuerst, dann Optionen, dann Empfehlung
Backbrief-Kata	20 Minuten	Einen empfangenen Auftrag mit eigenen Worten samt Absicht zurückerkennen

C.4 Kritische Würdigung

Martins Ton ist bewusst apodiktisch – „clean“ impliziert, dass Abweichung „schmutzig“ ist. Das hat der Branche einfache, lehrbare Regeln gegeben (seine Stärke als Doktrin), verleitet aber zu Dogmatik: Regeln ohne Kontextabwägung erzeugen Cargo-Kult (z. B. Zwang zu Kleinstfunktionen oder Schichten, wo keine nötig sind). Die produktive Lesart dieser Reihe: **als Doktrin nutzen, mit Carduccis „Tailor-Made“-Korrektiv lesen** (EA-Denkschrift A.2) – gemeinsame Regeln zuerst beherrschen, dann begründet abweichen. Genau so behandeln Militärs ihre Doktrin: Auftragstaktik erlaubt Abweichung, aber erst nach Beherrschung der Form.

BEDEUTUNG FÜR LAGEBILD & STABSARBEIT

(1) **Boundary-Reviews**: Solution-/ART-Schnitte regelmäßig gegen die Dependency Rule prüfen – Abhängigkeitsrichtung ist messbar (Dependency-Heatmap). (2) **Qualität als Geschwindigkeit** begründet die NFR-Dimension des Lagebilds ökonomisch. (3) **Planungs-Katas** zwischen den Curriculum-Modulen machen Übung zur Routine statt zum Event.

Teil D – Luke Hohmann: der Brückenbauer zum Geschäftsmodell

Person und Werklinie: Hohmann verbindet drei Karrieren: Software-Ingenieur und Architekturautor (*Beyond Software Architecture*, 2003, erschienen in Fowlers Signature Series), Unternehmer (Gründer von **Conteneo**, dessen Participatory-Budgeting-Plattform Portfolio-Entscheidungen in Konzernen skalierte – Firma aufgebaut und verkauft) und Methodenautor (*Innovation Games*, 2006; *Software Profit Streams™*, 2023, mit Jason Tanner und Federico González bei Applied Frameworks). Er war Beitragender zum SAFe-Framework – das dortige **Participatory Budgeting** im Lean Portfolio Management geht auf seine Methodik zurück.

D.1 Beyond Software Architecture – Tarchitecture trifft Marketecture

D.1.1 Die zwei Architekturen

Hohmanns Begriffspaar: Die „**Tarchitecture**“ (technische Architektur: Komponenten, Schnittstellen, Technologien) und die „**Marketecture**“ (Geschäftsarchitektur: Lizenzmodell, Preisstruktur, Zielsegmente, Deployment-, Upgrade- und Support-Modell, Branding) beschreiben *dasselbe Produkt* – und scheitern, wenn sie getrennt entworfen werden. Das Buch behandelt die vernachlässigte Hälfte: Lizenzierung und Lizenzdurchsetzung, Installation und Upgrade als Architekturthema (wer nicht upgraden kann, kauft nichts nach), Portabilität als Geschäftsentscheidung statt Technikreflex, Marken-/Namensfragen. Kernsatz sinngemäß: **Die beste technische Architektur ist wertlos, wenn die Geschäftsarchitektur nicht trägt – und umgekehrt.**

D.2 Innovation Games – ernsthafte Spiele als Entscheidungs-Rituale

Zwölf „ernsthafte Spiele“ für Produkt- und Portfolioentscheidungen, jeweils mit klarer Frage, Ablauf und Auswertung. Die für Portfolio-Stabsarbeit wichtigsten:

Spiel	Welche Frage es beantwortet	Einsatz im Portfolio-Kontext
Buy a Feature	Was ist es den Stakeholdern <i>wirklich</i> wert? (begrenztes Spielbudget, gemeinsames Kaufen; teure Features erzwingen Koalitionen)	Feature-/Epic-Priorisierung mit echten Stakeholdern; Übungsformat in Curriculum-Modul 2
Prune the Product Tree	Wo soll das Produkt wachsen – und wo nicht? (Baum-Metapher: Äste = Fähigkeiten)	Roadmap-Gespräche; ziviler Verwandter der Capability-Map
Speed Boat	Was bremst uns? (Anker am Boot benennen statt Personen kritisieren)	Impediment-/Schulden-Erhebung ohne Schuldzuweisung – Zulieferung fürs Lagebild
Remember the Future	Woran werden wir Erfolg erkannt haben? (aus der Zukunft rückwärts erzählen)	Zielbild-/Intent-Formulierung; verwandt mit Premortem (nur positiv gewendet)
Product Box	Was verspricht das Produkt – in Kundenworten? (Verpackung gestalten lassen)	Wertversprechen je Capability schärfen – Input für die Business-Lesart
20/20 Vision	Was ist wichtiger als was? (paarweises Ranking wie beim Optiker)	Schnelles Prioritäten-Ranking in Gremien, bevor WSJF-Zahlen gerechnet werden

D.2.1 Participatory Budgeting – das skalierte Ritual

Die Portfolio-Skalierung von Buy a Feature: Viele Beteiligte erhalten **Anteile eines realen Budgets** und verteilen es gemeinsam über konkurrierende Vorhaben – in moderierten Runden, in denen Koalitionen, Trade-offs und Begründungen *öffentlich* werden. Ergebnis ist nicht nur eine Rangfolge, sondern **geteiltes Verständnis und getragene Legitimität**: Wer mitverteilt hat, trägt die Entscheidung mit. SAFe hat die Methode als Standardpraxis für Lean Budgets übernommen (empfohlen im Halbjahres-/PI-Rhythmus). Für unsere Reihe ist Participatory Budgeting *das* Beispiel eines Rituals, das Shared Consciousness und Empowered Execution gleichzeitig erzeugt – McChrystals Kopplung, als Budgetprozess operationalisiert.

D.3 Software Profit Streams™ – Profitabilität wird entworfen

D.3.1 Die Kernideen (mit Jason Tanner, 2023)

- **Profitabilität ist eine Designdisziplin**: Nachhaltiger Gewinn entsteht nicht als Nebenprodukt guter Technik, sondern wird entlang des Lebenszyklus entworfen – mit dem **Profit Stream Canvas** als Arbeitsformat.
- **Drei Säulen**: (a) *Wert für den Kunden* quantifizieren (Kunden-ROI – ohne belegbaren Kundennutzen kein tragfähiger Preis), (b) *Wertabschöpfung gestalten* (Preismodell, Preispunkte, Packaging, Lizenzmodell samt Compliance/Durchsetzung), (c) *Nachhaltigkeit über den Lebenszyklus* (Upgrades, Wartung, Ökosystem – die Brücke zurück zu Beyond Software Architecture).
- **Lizenz- und Preisentscheidungen sind Architekturentscheidungen**: Metering, Mandantenfähigkeit, Deployment-Modell und Compliance-Nachweis müssen technisch getragen werden – Tarchitecture und Marketecture, zwanzig Jahre später zu Ende gedacht.

BEDEUTUNG FÜR LAGEBILD & STABSARBEIT

(1) Die **Business-Lesart** des Zwei-Lesarten-Layouts bekommt Substanz: Profit-/Kostenströme je Capability gehören perspektivisch ins Lagebild – sonst zeigt es nur die Kostenhälfte der Wahrheit. (2) **Entscheidungs-Rituale sind einkaufbar**: Buy a Feature und Participatory Budgeting stehen als Programme bereit (ToT D.1) – der schnellste Weg, Priorisierung vom Flurfunk in ein Ritual zu überführen. (3) **Speed Boat und Remember the Future** liefern moderierte Eingabeformate für Risiken/Schulden bzw. Intent – leichtgewichtige Quellen für die Phase-B-Artefakte der Roadmap.

Teil E – Synthese: vier Wege, ein Ziel

E.1 Wo die vier konvergieren

Vordenker	Kernbeitrag (eine Zeile)	Stützt in der EA-Denkschrift ...	Stützt in der ToT-Denkschrift ...
Hohpe	Übersetzung als Rolle; Optionen und Änderungsraten als Steuerungsgrößen	Befund 3 (Rosetta Stone), Prinzip „Zwei Lesarten“; Runway (Plattform)	Rollen-Mapping C.1 (vertikale Liaison); Kompetenzfeld 1 (Trends ins Lagebild)
Fowler	Evolution statt Masterplan; Muster als gemeinsames Vokabular	Prinzip 2 (Schulden sind Erstklass-Bürger); B.2 (Modernisierungs-Roadmaps)	Erfolgsfaktor 4 (Prozess-Standard); Doktrin-These B.3.6
Martin	Grenzen und Abhängigkeitsrichtung; Disziplin als Geschwindigkeit	Befund 4 (Kompetenz), Prinzip 2 (NFR/Qualität)	Didaktik-These (übungsbasiert); Curriculum-Katas
Hohmann	Geschäftsmodell und Technik zusammen entwerfen; Entscheidung als Ritual mit Beteiligung	Befund 3 und 5 (Business-Lesart, Software-Konsequenz)	Erfolgsfaktor 1+2 (Kopplung, Rituale vor Tools); B.3 Planspiele

Die gemeinsame Linie in vier Merksätzen: **Übersetze permanent** (Hohpe) · **verbessere evolutionär und benenne, was du tust** (Fowler) · **halte Grenzen sauber und übe die Form** (Martin) · **entscheide gemeinsam und entwirf auch das Geschäftsmodell** (Hohmann). Zusammen ergeben sie das zivile Fundament dessen, was die Reihe militärisch hergeleitet hat: Lagebild + Doktrin + Übung + dezentrale, legitimierte Entscheidung.

E.2 Ableitungen für unsere Software (SituationReport)

- 1 Trend-/Ableitungsspalten je Kennzahl (Hohpe – „first derivative“)**
Jede gepoolte Kennzahl erhält Richtung und Beschleunigung (Δ zum Vor-PI, Trendpfeil, Kippunkt-Schwelle) – berechenbar aus vorhandenen Jira-Daten: **Phase A** der Roadmap.
- 2 Schulden-Quadrant im NFR-/Schulden-Register (Fowler)**
Das Phase-B-Artefakt „NFR-/Runway-Register“ bekommt je Eintrag die Klassifikation bewusst/unbewusst × klug/leichtsinnig plus Tilgungsvermerk – Schulden werden steuerbar berichtet statt moralisch diskutiert.
- 3 Strangler-Fortschrittsmetrik für Modernisierungs-Epics (Fowler)**
Modernisierungs-Epics berichten „Anteil umgeleiteter Funktionalität/Traffic“ als Fortschrittsmaß (statt „% fertig“) – ein ehrlicher, messbarer Indikator; Eingabe leichtgewichtig: **Phase B**.
- 4 Abhängigkeitsrichtungs-Check in der Dependency-Heatmap (Martin – Dependency Rule)**
Die geplante Dependency-Heatmap markiert Abhängigkeiten, die „nach außen“ zeigen (Fachlichkeit hängt an Technik/Zulieferer) – Boundary-Verletzungen werden sichtbar, bevor sie teuer werden: **Phase B/C**.
- 5 Profit-Stream-Feld je Capability (Hohmann)**
Die Capability-Map (Phase-B-Artefakt) erhält optional Wertbeitrags-/Kostenstrom-Felder je Capability – die Business-Lesart bekommt eigene Daten statt nur umformulierter Flow-Metriken.

6 Ritual-Ergebnisse als Entscheidungs-Log-Import (Hohmann – Buy a Feature / Participatory Budgeting)

Ergebnisse von Priorisierungs-Sessions (Rangfolge, Budgetverteilung, Begründungen) fließen strukturiert ins Decision-/Assumption-Log – das Lagebild dokumentiert nicht nur *was* entschieden wurde, sondern *wie legitimiert*:
Phase B.

E.3 Leseptide nach Rolle

- **Portfolio-/Solution-Manager:** Hohpe *Elevator* → Hohmann *Beyond Software Architecture* (Teil Marketecture) → Fowler-bliki „Is High Quality Software Worth the Cost?“ → Hohmann/Tanner *Software Profit Streams*.
- **Lead-/Solution-Architekt:** Martin *Clean Architecture* → Hohpe/Woolf *EIP* (Katalogteil als Nachschlagewerk) → Fowler *PoEAA* + *Refactoring* → Hohpe *Platform Strategy*.
- **STE / PMO / Stabsrollen:** Hohmann *Innovation Games* → Hohpe *Elevator* (Kapitel Kommunikation) → Fowler-bliki „StranglerFigApplication“ → diese Nebenschrift Teil E als Landkarte.

Anschlussfähigkeit an die Reihe (Merkposten)

1

Team-of-Teams-Denkschrift (v3.0)

Teil C.4 ist die Kurzfassung dieser Nebenschrift aus Stabs-/Führungssicht; die Planungs-Katas (hier C.3) und die Programmzeile *Innovation Games* (dort D.1) verzahnen beide Dokumente.

2

EA-Denkschrift (v2.0)

Teil A2 enthält die EA-gerichteten Kurzprofile; die fünf Lagebild-Prinzipien von dort werden hier je Autor untermauert und in Teil E auf Software-Features heruntergebrochen.

3

Lagebild-Literaturliste (Ergänzung v3)

Dort die Kurzwürdigungen aller acht Werke mit LS-Fokus-Markierung; diese Nebenschrift ist die Detailausarbeitung dazu – die Software-Roadmap bleibt in `docs/portfolio_roadmap_solution-lagebild.md`.

Quellenverzeichnis

Gregor Hohpe: Hohpe/Woolf: *Enterprise Integration Patterns – Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003. · *The Software Architect Elevator – Redefining the Architect's Role in the Digital Enterprise*. O'Reilly, 2020. · *Cloud Strategy – A Decision-Based Approach to Successful Cloud Migration*. Architect Elevator Series, 2020. · Hohpe/Danieli/Landreau: *Platform Strategy – Innovation Through Harmonization*. Architect Elevator Series, 2024. · Web: architectelevator.com.

Martin Fowler: *Refactoring – Improving the Design of Existing Code*, 2. Aufl. Addison-Wesley, 2018 (Erstauflage 1999). · *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. · bliki-Artikel (martinfowler.com): „StranglerFigApplication“ (2004), „TechnicalDebtQuadrant“ (2009), „DesignStaminaHypothesis“, „MonolithFirst“, „MicroservicePrerequisites“, „BranchByAbstraction“, „Yagni“, „Is High Quality Software Worth the Cost?“ (2019).

Robert C. Martin: *Clean Code – A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. · *Clean Architecture – A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

Luke Hohmann: *Beyond Software Architecture – Creating and Sustaining Winning Solutions*. Addison-Wesley, 2003. · *Innovation Games – Creating Breakthrough Products Through Collaborative Play*. Addison-Wesley, 2006. · Tanner/Hohmann (mit Federico González): *Software Profit Streams™ – A Guide to Designing a Sustainably Profitable Business*. Applied Frameworks, 2023. · SAFe-Guidance „Participatory Budgeting“ (auf Hohmanns/Conteneos Methodik beruhend).

Interne Bezüge: `Team-of-Teams-und-Stabsausbildung_v3.0.md` (Teil C.4) · `Enterprise-Architektur-und-Solution-Lagebild_v2.0.md` (Teil A2) · `Lagebild-Literaturliste.md` (Ergänzung v3) · `docs/portfolio_roadmap_solution-lagebild.md` (Phasen A–C).

Versionshistorie

Version	Datum	Änderungen
1.0	07.08.2026	Erstfassung der Nebenschrift: vier Detail-Profile (Hohpe: Elevator/EIP-Musterauswahl/Cloud- & Platform-Strategy; Fowler: Refactoring/Schulden-Quadrant/PoEAA-Auswahl/Strangler Fig & bliki; Robert C. Martin: Clean Code/Dependency Rule/SOLID/Katas & kritische Würdigung; Hohmann: Tarchitecture-Marketecture/Innovation-Games-Katalog/Participatory Budgeting/Profit Streams); Synthese mit Konvergenztabelle, sechs Software-Ableitungen (Roadmap-Phasen A–C) und Lesepfaden; drei Abbildungen; DE + EN.

Quellen-Briefing „Vier Architektur-Vordenker im Detail“ (Nebenschrift) · Version 1.0 · Solution-/Portfolio-Management · Stand 07.08.2026 · kuratiert von Robert Seebauer, erstellt mit Claude Fable 5 (Anthropic) · Markdown-Quelle: [Quellen/Architektur-Vordenker_v1.0.md](#)